

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO
DEPARTAMENTO DE COMPUTAÇÃO

Marcelo da Silva Zanata – 16831070

Relatório

Busca e Organização de Dados

SÃO CARLOS – SP
2026

1. Organização do Código e Módulos

O projeto foi construido seguindo as boas práticas de encapsulamento e modularização de Tipos Abstratos de Dados (TADs), mantendo isolamento absoluto entre lógica dos dados, leitores e estruturas de armazenamento, gostaria de comentar um pouco a respeito da implementação do “ITEM”, o parâmetro reader ele é descartado no código (não é necessário, dado que foi generalizado o leitor), pois fiz a implementação baseada na forma que eu acredito que a biblioteca Pandas do Python realiza leitura de CSVs inferindo os respectivos tipos de dados gerando um dataframe (uma estrutura de dados genérica e adaptável).

A busca, no caso a função de match, é totalmente compátivel com o tipo de dado genérico implementado. A implementação seguiu ideias intuitivas como utilizar o recurso UNION do C, para ser possível ter um campo que possa assumir todos os tipos de dados (int, string, float, bool, null), desse modo, a partir da concepção do que seria uma estrutura de dados genérica, foi trabalhar no parser para ler corretamente o arquivo CSV e ser capaz de inferir corretamente o tipo de dado, mais uma vez, simplesmente na tentativa de replicar o dataframe do Pandas.

Essa abordagem de generalizar para qualquer CSV vem da ideia de ter um software realmente útil, não estático em apenas um ou dois formatoa de CSV, e que, ainda sim, poderia enfrentar diversos problemas de implementação por ter readers e buscas separados.

A respeito da implementação das estruturas de dados, elas foram implementadas da mesma forma que foram ensinadas em sala de aula.

Tabela 1: Árvore de Diretórios do Projeto

```
├── Makefile
├── c_project/
│   ├── Makefile
│   ├── include/
│   │   ├── generic_item.h
│   │   ├── generic_search.h
│   │   ├── estrutura.h
│   │   ├── reader.h
│   │   ├── pilha.h
│   │   ├── fila.h
│   │   ├── lista_seq.h
│   │   ├── lista_enc.h
│   │   ├── lista_cabeca.h
│   │   ├── lista_ord.h
│   │   ├── lista_gen.h
│   │   └── lista_cruz.h
│   └── src/
├── dados/
│   ├── asteroids.csv
│   └── mortes.csv
└── consultas/
    ├── q1.txt, q2.txt, q3.txt
    └── q_mortes1.txt, q_mortes2.txt, q_mortes3.txt
```

2. Implementação das 8 Estruturas de Dados (TADs)

2.1. Pilha (pilha.c): Estrutura dinâmica encadeada com disciplina *LIFO* (Last In, First Out). Possui ponteiro para o topo e contagem de elementos. No desempilhamento para exportação, o elemento mais recente é processado primeiro.

2.2. Fila (fila.c): Estrutura dinâmica com disciplina *FIFO* (First In, First Out), mantendo ponteiros de acesso O(1) para inicio e fim.

2.3. Lista Sequencial (lista_seq.c): Armazenamento contínuo em vetor dinâmico de ponteiros para itens com política de duplicação geométrica de capacidade (realloc), garantindo custo O(1) na inserção.

2.4. Lista Encadeada (lista_enc.c): Lista encadeada linear com nós contendo ponteiro para o item e ponteiro para o próximo nó.

2.5. Lista com Cabeça (lista_cabeca.c): Atende rigorosamente ao requisito de possuir um nó cabeça/sentinela real alocado em memória (cabeca->item == NULL). Todas as operações de inserção, remoção e percurso utilizam essa sentinela, eliminando casos especiais para listas vazias.

2.6. Lista Ordenada (lista_ord.c): Atende estritamente à regra de que a ordenação é realizada durante a inserção (custo O(N) por item, total $O(N^2)$ para N itens), nunca inserindo desordenado para ordenar ao final. Suporta campos numéricos, alfanuméricos e o pseudocampo especial (similarity).

2.7. Lista Generalizada (lista_gen.c): Estrutura multinível hierárquica que agrupa registros dinamicamente em dois níveis conforme o comando GROUP (ex: threat_category -> year). Cada grupo possui uma sublista de subgrupos, que por sua vez contém a lista encadeada de itens.

2.8. Lista Cruzada (lista_cruz.c): Matriz esparsa ortogonal em que o mesmo registro participa de duas dimensões simultaneamente sem duplicação de memória. Cada nó (NoCruzado) contém dois ponteiros independentes de encadeamento: prox_eixo1 e prox_eixo2, ligados aos respectivos cabeçalhos de eixo. A liberação de memória é realizada apenas por um dos eixos com auxílio de contagem de referências no ITEM.

3. Gestão de Memória e Compartilhamento de Registros

Para evitar vazamentos de memória e ao mesmo tempo permitir que a estrutura (base) e a estrutura (result) referenciem os mesmos registros (ou subconjuntos deles):

- O tipo abstrato ITEM implementa contagem de referências (item->refs).
- Ao ser carregado do CSV para a base, o item é instanciado com refs = 1.
- Ao ser aprovado no filtro da busca e inserido em (result), a função item_retain(item) incrementa o contador para 2.

- Quando (base) ou (result) são destruídas, a função item_free(item) decrementa o contador, liberando buffers de texto e estrutura apenas quando refs == 0.
- Isso foi validado com AddressSanitizer (-fsanitize=address,undefined), garantindo zero vazamentos de memória e zero acessos inválidos.

4. Seis Exemplos Completos de Execução

Exemplo 1 (Parte 1 - NASA: Lista Encadeada -> Lista Ordenada por Similaridade)

Comando:

```
./trabalho --reader nasa --input dados/asteroids.csv --base lista_enc --result lista_ord --query consultas/q1.txt --output res_ex1.csv --top-k 5 --limit 1000
```

Consulta (consultas/q1.txt):

```
FILTER|distance_au|<=|0.08
SIM|velocity_km_s|15.0|10.0|1.0
SIM|risk_score|0.70|0.30|2.0
ACCEPT|0.60
ORDER|similarity|DESC
GROUP|threat_category|year
CROSS|threat_category|year
```

Saída Padrão:

```
STATUS=OK
INPUT_RECORDS=1000
MATCHED_RECORDS=10
OUTPUT_RECORDS=5
LOAD_MS=5.832
SEARCH_MS=0.225
RESULT_INSERT_MS=0.002
WRITE_MS=0.141
```

Resultado Obtido: Os primeiros 5 registros em res_ex1.csv correspondem aos asteroides com maior proximidade dos valores ideais de velocidade (15 km/s) e risco (0.70), ordenados em ordem decrescente de similaridade (liderados por 2020FG e 2020FD).

Exemplo 2 (Parte 1 - NASA: Pilha -> Fila com Lista Sentry)

Comando:

```
./trabalho --reader nasa --input dados/asteroids.csv --base pilha --result fila --query consultas/q2.txt --output res_ex2.csv --top-k 10 --limit 1000
```

Consulta (consultas/q2.txt):

```
FILTER|on_sentry_list|==|true
SIM|sentry_diameter_km|0.50|0.50|1.0
ACCEPT|0.30
ORDER|sentry_diameter_km|DESC
GROUP|threat_category|year
CROSS|threat_category|year
```

Saída Padrão:

```
STATUS=OK
INPUT_RECORDS=1000
MATCHED_RECORDS=0
OUTPUT_RECORDS=0
LOAD_MS=4.744
SEARCH_MS=0.168
RESULT_INSERT_MS=0.000
WRITE_MS=0.096
```

Resultado Obtido: Nada foi encontrado, logo, o csv resultante possui apenas cabeçalho.

Exemplo 3 (Parte 1 - NASA: Lista com Cabeça -> Lista Cruzada para Eventos Passados)

Comando:

```
./trabalho --reader nasa --input dados/asteroids.csv --base lista_cabeca --result lista_cruz --query consultas/q3.txt --output res_ex3.csv --top-k 0 --limit 0
```

Consulta (consultas/q3.txt):

```
FILTER|is_past_event|==|true
FILTER|distance_au|<=|0.05
ORDER|close_approach_date|DESC
GROUP|panic_level|year
CROSS|threat_category|year
```

Saída Padrão:

```
STATUS=OK
INPUT_RECORDS=89227
MATCHED_RECORDS=9382
OUTPUT_RECORDS=9382
LOAD_MS=354.992
SEARCH_MS=8.882
RESULT_INSERT_MS=5.523
WRITE_MS=31.891
```

Resultado Obtido: Demonstra o percurso ortogonal da Lista Cruzada indexada simultaneamente por threat_category e year. A saída contém diversos asteróides que foram compativeis com os filtros.

Exemplo 4 (Parte 2 - Dados Abertos: Lista Encadeada -> Lista Ordenada)

Comando:

```
./trabalho --reader mortes --input dados/mortes.csv --base lista_enc --result lista_ord --query consultas/q_mortes1.txt --output res_ex4.csv --top-k 50 --limit 10000
```

Consulta (consultas/q_mortes1.txt):

```
FILTER REGIAO|==|Interior
FILTER ANO|==|2024
```

```
FILTER QT_VITIMAS|>=|1
ORDER QT_VITIMAS|DESC
```

Saída Padrão:

```
STATUS=OK
INPUT_RECORDS=5173
MATCHED_RECORDS=1551
OUTPUT_RECORDS=50
LOAD_MS=9.922
SEARCH_MS=0.752
RESULT_INSERT_MS=107.460
WRITE_MS=0.158
```

Resultado Obtido: Os 50 municípios com os maiores números de vítimas de assassinato.

Exemplo 5 (Parte 2 - Dados Abertos: Fila -> Lista Encadeada para Lista Ordenada)

Comando:

```
./trabalho --reader mortes --input dados/mortes.csv --base lista_enc --result lista_ord
--query consultas/q_mortes2.txt --output res_ex5.csv --top-k 10 --limit 10000
```

Consulta (consultas/q_mortes2.txt):

```
FILTER REGIAO|==|Interior
SIM QT_VITIMAS|2.0 5.0 1.0
SIM MES|1.0 12.0 0.5
ACCEPT 0.50
ORDER similarity|DESC
```

Saída Padrão :

```
STATUS=OK
INPUT_RECORDS=5173
MATCHED_RECORDS=4351
```

```
OUTPUT_RECORDS=10
LOAD_MS=10.299
SEARCH_MS=1.012
RESULT_INSERT_MS=78.498
WRITE_MS=0.202
```

Resultado Obtido: Filtra os assassinatos no interior com o top 10.

Exemplo 6 (Parte 2 - Dados Abertos: Lista com Cabeça -> Lista Ordenada)

Comando:

```
./trabalho --reader mortes --input dados/mortes.csv --base lista_cabeca --result lista_ord --query consultas/q_mortes3.txt --output res_ex6.csv --top-k 0 --limit 0
```

Consulta (consultas/q_mortes3.txt):

```
FILTER ANO|==|2024
GROUP GR_NATUREZA|REGIAO
CROSS GR_NATUREZA REGIAO
ORDER QT_VITIMAS|DESC
```

Saída Padrão:

```
STATUS=OK
INPUT_RECORDS=5173
MATCHED_RECORDS=1730
OUTPUT_RECORDS=1730
LOAD_MS=9.030
SEARCH_MS=0.573
RESULT_INSERT_MS=131.128
WRITE_MS=1.526
```

Resultado Obtido: Filtra os assassinatos no ano de 2024 com a quantidade de vitimas decrescente.